Clasificación de personajes de Los Simpson con redes convolucionales

Práctica 1 · Aprendizaje Automático II · Curso 2026-27

Grado en Ciencia e Ingeniería de Datos · ULPGC



Autores: Aimar Daniel Alejandro Santana · Pablo Martínez Suárez

31 de octubre de 2026

Resumen. Se ha desarrollado en PyTorch un clasificador de 10 personajes de Los Simpson basado en redes convolucionales con *pooling*, *dropout* y parada anticipada. Se probaron 7 configuraciones manuales (C1–C7), 2 con aumentado de datos (C8–C9), una búsqueda automática con Optuna de 25 pruebas cuyo ganador se reentrenó con aumento (C10) y una ResNet18 preentrenada con *fine-tuning* (C11). El mejor modelo, elegido por la menor pérdida de validación, es **C10**: **95,4 % de accuracy en validación** y **100 % en el conjunto de test** (496 imágenes). En la sección 8 se explica por qué el test de Kaggle sobreestima el rendimiento real y por qué consideramos el 95,4 % como la estimación fiable.

1. Problema y conjunto de datos

Hemos elegido el conjunto *The Simpsons Characters Data* (Kaggle, autor alexattia). Contiene fotogramas de la serie recortados y etiquetados por personaje. Nos quedamos con los **10 personajes con más imágenes**, lo que cumple el requisito de 5 a 15 clases con al menos 50 imágenes cada una (la menos representada, Milhouse, tiene 1.079). Es un problema más difícil que los *toy problems* tipo MNIST o CIFAR: las imágenes tienen tamaños, encuadres y fondos muy distintos, aparecen otros personajes en segundo plano y varios personajes comparten rasgos (piel amarilla, mismo estilo de dibujo).

El conjunto de entrenamiento original (13.811 imágenes) se divide de forma **estratificada** en 85 % entrenamiento y 15 % validación con `train_test_split(..., stratify=etiquetas, random_state=42)`. La estratificación mantiene la misma proporción de cada personaje en los dos subconjuntos, algo importante porque las clases no están equilibradas (Homer tiene el doble de imágenes que Milhouse). Como test se usa el conjunto de test oficial que acompaña al dataset, que solo se evalúa una vez al final.

Personaje	Entrenamiento	Validación	Test
Homer Simpson	1909	337	50
Ned Flanders	1236	218	49
Moe Szyslak	1234	218	50
Lisa Simpson	1151	203	50
Bart Simpson	1141	201	50
Marge Simpson	1097	194	50
Krusty The Clown	1025	181	50
Principal Skinner	1015	179	50
Charles Montgomery Burns	1014	179	48
Milhouse Van Houten	917	162	49
Total	**11.739**	**2.072**	**496**

Tabla 1. Imágenes por personaje y conjunto.

Preprocesado

Cada imagen se convierte a RGB, se redimensiona a **128×128 píxeles** y se pasa a un tensor con forma (canales, alto, ancho) y valores en [0, 1] (división entre 255). 128 px es un compromiso: con 64 px se pierden detalles como el pelo de Lisa o las gafas de Skinner, y con 224 px el entrenamiento sería mucho más lento sin una mejora clara. Las imágenes se guardan en caché en formato `.npy` para no tener que leer los JPG en cada ejecución, y se cargan con `TensorDataset` y `DataLoader` (barajado solo en entrenamiento).

2. Arquitectura y entrenamiento

Definimos una clase `ConvNet` configurable para poder probar distintas arquitecturas con el mismo código. Cada bloque convolucional aplica **convolución → función de activación → max pooling 2×2**, de modo que el lado de la imagen se divide entre 2 en cada bloque. Después se aplana el resultado y se pasa por una o varias capas totalmente conectadas, cada una seguida de **dropout**, y una capa de salida con 10 neuronas (una por personaje). Las capas se guardan en `nn.ModuleList`, así el número de capas convolucionales y de capas FC es un hiperparámetro más. El *padding* es `kernel // 2`, con lo que la convolución mantiene el tamaño y solo el pooling lo reduce.

Capa (C1)	Forma de salida
Entrada	3 × 128 × 128
Conv 5×5 (16) + ReLU + MaxPool	16 × 64 × 64
Conv 5×5 (32) + ReLU + MaxPool	32 × 32 × 32
Conv 5×5 (64) + ReLU + MaxPool	64 × 16 × 16
Aplanado (64 × 16 × 16)	16.384 valores
FC 128 + ReLU + Dropout	128
Salida	10

Tabla 2. Arquitectura de la configuración base C1 (2,16 millones de parámetros).

Todas las configuraciones se entrenan con la función de pérdida `CrossEntropyLoss` (combina softmax y log-verosimilitud negativa) y el optimizador **Adam**. La semilla se fija a 42 para que los experimentos sean comparables.

Función de pérdida y optimizador

La red devuelve un vector z de 10 valores (*logits*), uno por personaje. `CrossEntropyLoss` los convierte en probabilidades con softmax y penaliza el logaritmo de la probabilidad asignada a la clase correcta y:

$$L = -\log\left(\, e^{z_y} / \Sigma_j\, e^{z_j} \,\right) = -z_y + \log \Sigma_j\, e^{z_j}$$

Si la red da una probabilidad alta al personaje correcto, la pérdida es casi 0; si se la da baja, la pérdida crece mucho. PyTorch usa la segunda forma de la ecuación (truco *log-sum-exp*), que evita desbordamientos numéricos cuando los logits son muy grandes.

Adam ajusta una tasa de aprendizaje distinta para cada peso. Para ello guarda una media móvil del gradiente g (momento m) y de su cuadrado (momento v):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)\, g_t$$
$$v_t = \beta_2 v_{t-1} + (1 - \beta_2)\, g_t^{\,2}$$
$$\theta_t = \theta_{t-1} - \eta \cdot \hat{m}_t \,/\, (\sqrt{\hat{v}_t} + \varepsilon)$$

donde η es la tasa de aprendizaje y $\hat{m}$, $\hat{v}$ son los momentos corregidos para que no estén sesgados hacia 0 al principio. Usamos los valores por defecto $\beta_1 = 0{,}9$, $\beta_2 = 0{,}999$ y $\varepsilon = 10^{-8}$. Los pesos que reciben gradientes grandes y ruidosos avanzan con pasos más pequeños, y por eso Adam converge más rápido que el descenso de gradiente simple.

Parada anticipada (early stopping)

Al final de cada época se calcula la pérdida en validación. Si mejora, guardamos una copia de los pesos (`copy.deepcopy(modelo.state_dict())`). Si no mejora durante un número de épocas seguidas (**paciencia**: 5 en la búsqueda manual, 8 con aumentado de datos), el entrenamiento se detiene y se restauran los pesos de la mejor época. Así el modelo final es el de menor pérdida de validación y no el de la última época, que normalmente ya estaría sobreajustado. Vigilamos la pérdida y no la accuracy porque es más sensible: refleja también la confianza de las predicciones y empeora antes cuando el modelo empieza a memorizar.

3. Búsqueda manual de hiperparámetros (C1–C7)

Partimos de una red pequeña y en cada configuración cambiamos uno o dos hiperparámetros respecto a la anterior, para poder atribuir cada mejora (o empeoramiento) a ese cambio. Máximo 50 épocas y paciencia 5.

Conf.	Filtros	FC	Activ.	Drop.	Kernel	LR	Lote	Parám.	Mejor ép.	Val Loss	Val Acc
C1	16-32-64	128	ReLU	0,25	5×5	0,001	32	2,16 M	5/10	0,638	81,7 %
C2	16-32-64	128	ReLU	0,5	5×5	0,001	32	2,16 M	6/11	0,567	82,9 %
C3	32-64-128	256	ReLU	0,5	3×3	0,001	32	8,48 M	6/11	0,547	84,9 %
C4	32-64-128-256	256	ReLU	0,5	3×3	0,001	32	4,59 M	8/13	0,405	89,7 %
C5	32-64-128-256	256	ReLU	0,5	3×3	0,0005	64	4,59 M	8/13	0,470	86,6 %
C6	32-64-128-256	256	LeakyReLU	0,5	3×3	0,0005	64	4,59 M	8/13	0,405	88,8 %
C7	32-64-128-256	512-128	ReLU	0,5	3×3	0,0005	64	8,84 M	11/16	0,384	89,3 %

Tabla 3. Configuraciones de la búsqueda manual. «Mejor ép.» indica la época restaurada por el early stopping sobre el total de épocas ejecutadas.

- **C1 (base, 81,7 %).** 3 capas convolucionales de 16, 32 y 64 filtros, kernel 5×5, una capa FC de 128 neuronas y dropout 0,25. Ya en la época 5 la pérdida de validación empieza a subir mientras la de entrenamiento sigue bajando: sobreajuste claro.
- **C2 (dropout 0,5, 82,9 %).** Más dropout para frenar el sobreajuste. La pérdida de validación mejora (0,64 → 0,57).
- **C3 (más filtros, kernel 3×3, 84,9 %).** Duplicamos los filtros y la capa FC. Mejora, pero el número de parámetros se dispara a 8,5 M, casi todos en la primera capa FC (64×16×16 → 32.768 entradas).
- **C4 (4 capas convolucionales, 89,7 %).** El salto más grande. El cuarto bloque reduce el mapa a 8×8, así que la red es más profunda pero tiene *menos* parámetros (4,6 M) y aprende rasgos más abstractos. Fijamos esta arquitectura convolucional.
- **C5 (LR 0,0005 y lote 64, 86,6 %).** Una tasa de aprendizaje menor y un lote mayor dan un entrenamiento más suave, pero en este caso algo peor.
- **C6 (LeakyReLU, 88,8 %).** Con LR 0,0005 y lote 64, cambiar ReLU por LeakyReLU recupera el nivel de C4 (misma pérdida de validación, 0,405). LeakyReLU deja pasar una pequeña parte de los valores negativos y evita neuronas «muertas».
- **C7 (2 capas FC 512-128, 89,3 %).** Una segunda capa totalmente conectada da la **menor pérdida de validación** de la búsqueda manual (0,384).

Conclusiones de la búsqueda manual: la profundidad convolucional es lo que más ayuda; el dropout alto (0,5) es necesario; y con 4 bloques convolucionales todas las variantes quedan en torno al 87–90 %. El problema común es el **sobreajuste**: en todas las curvas la accuracy de entrenamiento supera el 90 % mientras la de validación se estanca. Esto nos llevó a probar el aumentado de datos.

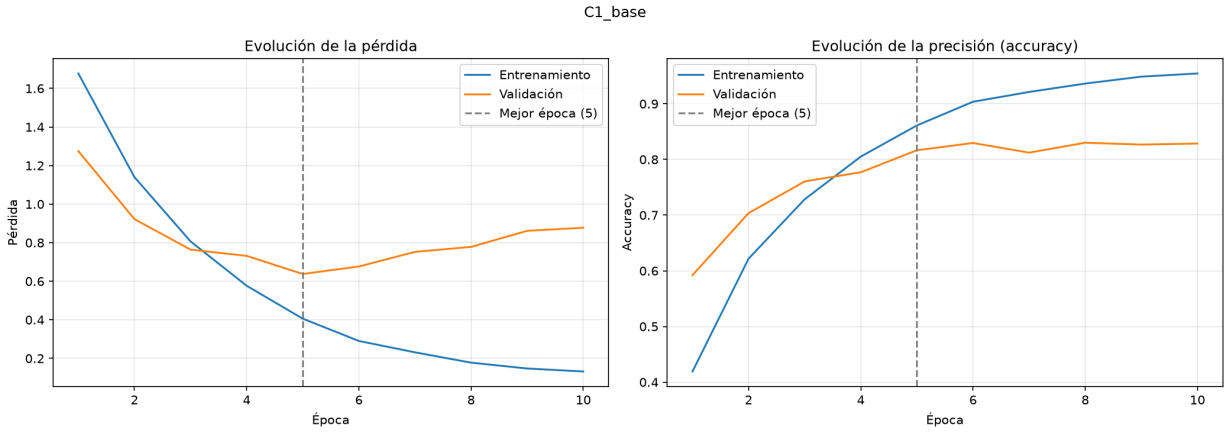


Figura 1. Curvas de pérdida y accuracy de C1. La línea discontinua marca la época restaurada por el early stopping (5); a partir de ahí la pérdida de validación sube.

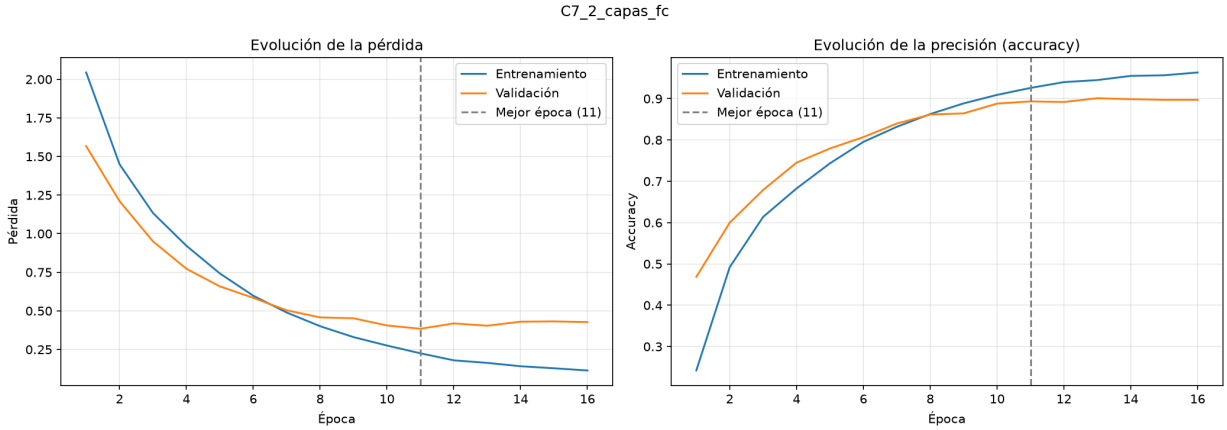


Figura 2. Curvas de C7, la configuración manual con menor pérdida de validación.

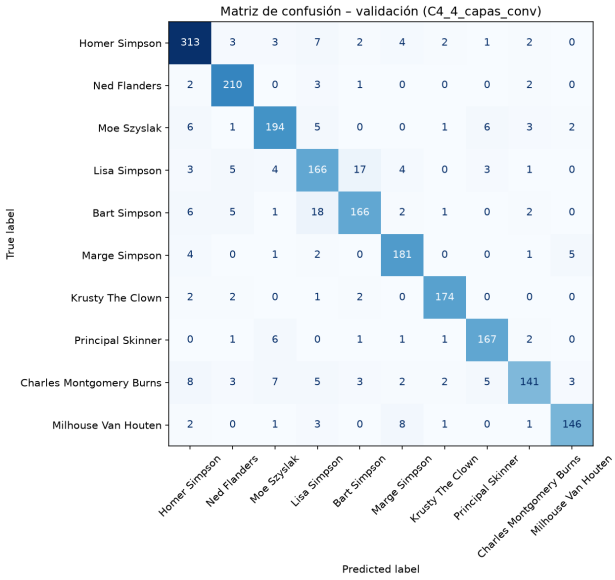


Figura 3. Matriz de confusión en validación de C4 (mejor accuracy manual, 89,7 %). Los errores se concentran en Lisa, Bart y Burns.

La matriz de confusión muestra que los personajes más difíciles son **Lisa** (F1 0,80), **Bart** (0,85) y **Burns** (0,84, con recall de solo 0,79). Lisa y Bart son hermanos con la misma forma de cabeza y colores parecidos; Burns se confunde con otros personajes de piel amarilla y pelo canoso. En cambio Krusty (0,96) y Ned Flanders (0,94) tienen rasgos muy característicos (maquillaje de payaso, bigote y gafas).

4. Aumentado de datos (C8–C9)

Para reducir el sobreajuste aplicamos transformaciones aleatorias a cada imagen de entrenamiento en cada época, de modo que la red nunca ve dos veces exactamente la misma imagen. Solo se aplican en entrenamiento; validación y test se evalúan con las imágenes originales.

- `RandomHorizontalFlip(p=0.5)`: volteo horizontal (un personaje mirando a la izquierda sigue siendo el mismo).
- `RandomRotation(10)`: giros de hasta ±10°.
- `RandomResizedCrop(128, scale=(0.8, 1.0))`: recortes de entre el 80 % y el 100 % del área (zoom y desplazamiento).
- `ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2)`: cambios de brillo, contraste y saturación. No variamos el tono (*hue*) porque el color es una pista clave: cambiarlo convertiría la piel amarilla en verde o el pelo de Marge en otro color.
- No usamos volteo vertical: los personajes nunca aparecen boca abajo, así que la red aprendería casos que no existen.



Figura 4. Una imagen original y 15 versiones aumentadas generadas aleatoriamente.

Aplicamos el aumento a las dos mejores configuraciones manuales, C6 (menor número de parámetros entre las mejores) y C7 (menor pérdida), con hasta 80 épocas y paciencia 8, porque con aumento la red tarda más en converger.

Conf.	Base	Mejor ép.	Val Loss	Val Acc	Mejora vs. sin aumento
C8	C6	14/22	0,258	93,3 %	+4.5 puntos
C9	C7	19/27	0,229	93,7 %	+4.3 puntos

Tabla 4. Resultados con aumentado de datos.

El aumento mejora claramente: unos **4–5 puntos de accuracy** y una pérdida de validación que baja de 0,38–0,41 a 0,23–0,26. Las curvas muestran que entrenamiento y validación van mucho más juntas y que la red entrena durante más épocas antes de que actúe el early stopping (22 y 27 frente a 13 y 16).

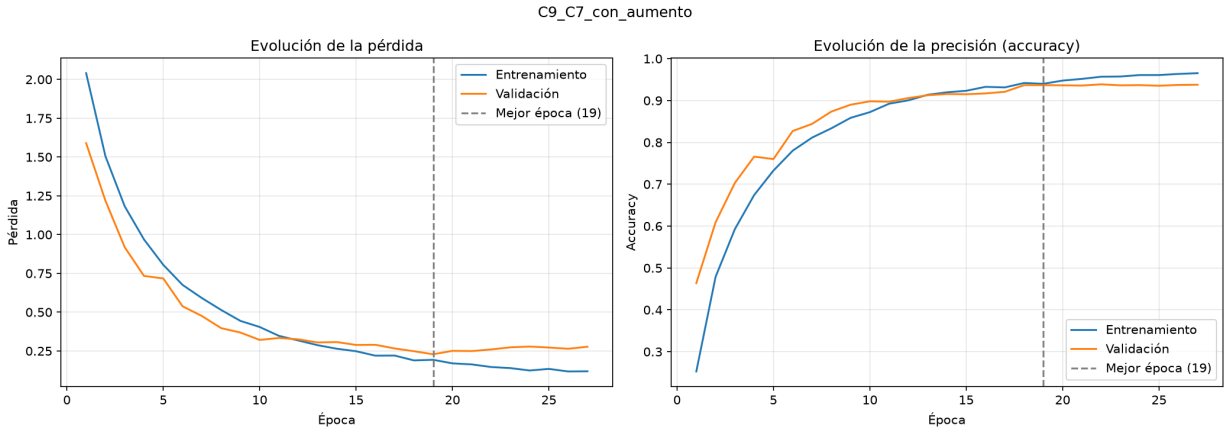


Figura 5. Curvas de C9 (C7 con aumentado de datos). La separación entre entrenamiento y validación es mucho menor que en la Figura 2.

5. Búsqueda automática de hiperparámetros con Optuna (C10)

Después de la búsqueda manual usamos **Optuna** para explorar el espacio de hiperparámetros de forma automática. Optuna usa un **muestreador TPE** (*Tree-structured Parzen Estimator*, una búsqueda bayesiana): separa las pruebas buenas de las malas y propone nuevas combinaciones parecidas a las buenas. Además, un **MedianPruner** poda las pruebas que, a partir de la época 3, van por debajo de la mediana de las anteriores, para no gastar tiempo en ellas. Cada prueba entrena hasta 25 épocas con paciencia 4 y **sin aumento** (para que la búsqueda sea rápida). El objetivo a maximizar es la accuracy de validación.

Hiperparámetro	Valores explorados	Mejor valor
Capas convolucionales	3 o 4	4
Filtros de la primera capa (se duplican en cada capa)	16, 32, 64	32 → (32, 64, 128, 256)
Tamaño del kernel	3, 5	3
Capas FC (las neuronas se dividen entre 2)	1 o 2	1
Neuronas de la primera FC	128, 256, 512	256
Función de activación	ReLU, LeakyReLU, ELU	LeakyReLU
Dropout	0,2 – 0,6 (continuo)	0,58
Tasa de aprendizaje	0,0001 – 0,003 (escala log.)	0,00065
Tamaño de lote	32, 64, 128	64

Tabla 5. Espacio de búsqueda de Optuna y mejor combinación encontrada (prueba 13).

Se ejecutaron **25 pruebas en 21,8 minutos**: 16 completas y 9 podadas. La mejor (prueba 13) alcanzó un **90,3 %** sin aumento, por encima de cualquier configuración manual sin aumento. La combinación encontrada se parece mucho a C6 (4 capas de 32 a 256 filtros, LeakyReLU, lote 64), pero con más dropout (0,58) y una LR algo mayor. Esto confirma que la búsqueda manual iba en la buena dirección. El análisis de importancia de Optuna señala el **dropout** como el hiperparámetro más influyente con diferencia, seguido del número de filtros y de capas convolucionales; el tamaño de lote y el número de capas FC apenas influyen.

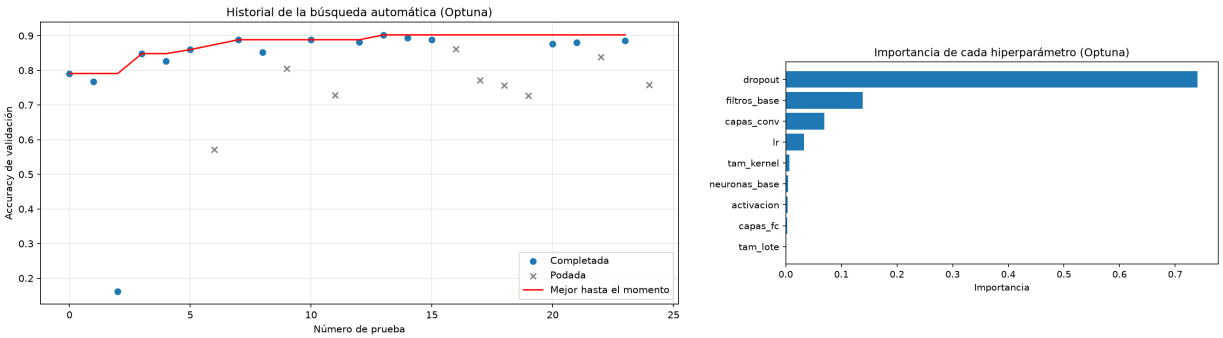


Figura 6. Izquierda: accuracy de cada prueba (x = podada) y mejor resultado acumulado. Derecha: importancia de cada hiperparámetro.

Finalmente reentrenamos la mejor combinación **con aumentado de datos**, hasta 80 épocas y paciencia 8: es la configuración **C10**. Obtiene **95.4 % de accuracy** y una pérdida de validación de **0.221** (mejor época 23 de 31), los mejores valores de todo el trabajo.

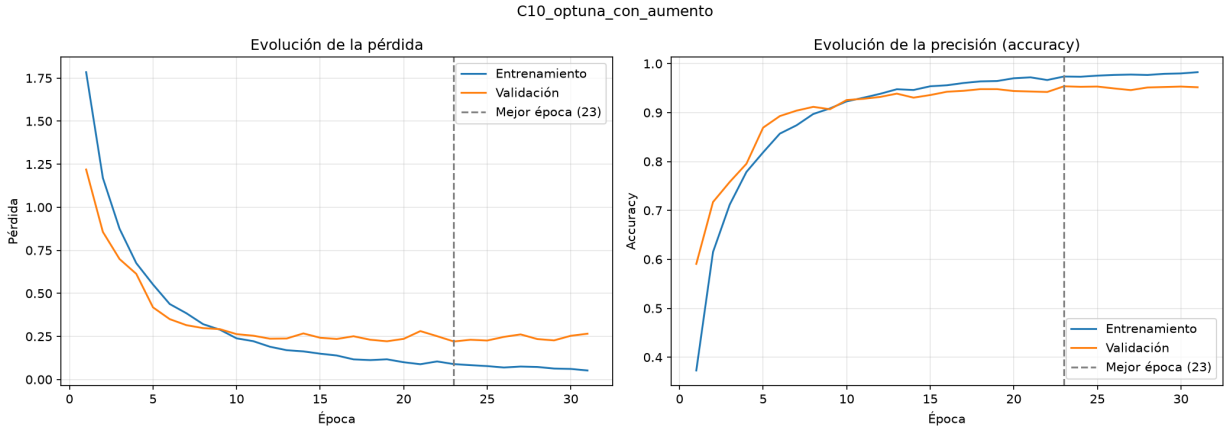


Figura 7. Curvas de C10. Las curvas de entrenamiento y validación se mantienen muy próximas durante todo el entrenamiento.

6. Transferencia de conocimiento con ResNet18 (C11)

Como última estrategia usamos una **ResNet18 preentrenada en ImageNet** (1,2 millones de fotografías, 1.000 clases). Sustituimos su última capa por `Dropout(0.5) + Linear(512, 10)` y hacemos un *fine-tuning* parcial: congelamos todas las capas salvo el último bloque residual (`layer4`) y la nueva capa de salida. Así se reentrenan 8,4 de sus 11,2 millones de parámetros. Las primeras capas, que detectan bordes y texturas genéricas, se conservan tal cual; las últimas se adaptan a los personajes. Como la red se entrenó con imágenes normalizadas con la media y la desviación de ImageNet, aplicamos esa misma normalización dentro del modelo, *después* del aumentado de datos. Entrenamos con LR 0,0005, lote 64, aumento, hasta 40 épocas y paciencia 6.

C11 converge muy rápido: **90,3 % ya en la primera época**, gracias al conocimiento previo. Pero se estanca en el **93,3 %** (pérdida 0,277, mejor época 7 de 13), por debajo de C10. Creemos que se debe a dos motivos: ImageNet son fotografías reales, mientras que nuestras imágenes son dibujos animados con colores planos y contornos negros, así que lo aprendido se transfiere solo en parte; y la ResNet18 está pensada para imágenes de 224 px, mientras que nosotros usamos 128 px. Además tiene tanta capacidad que sobreajusta enseguida: la accuracy de entrenamiento llega al 97 % mientras la pérdida de validación deja de mejorar casi desde el principio.

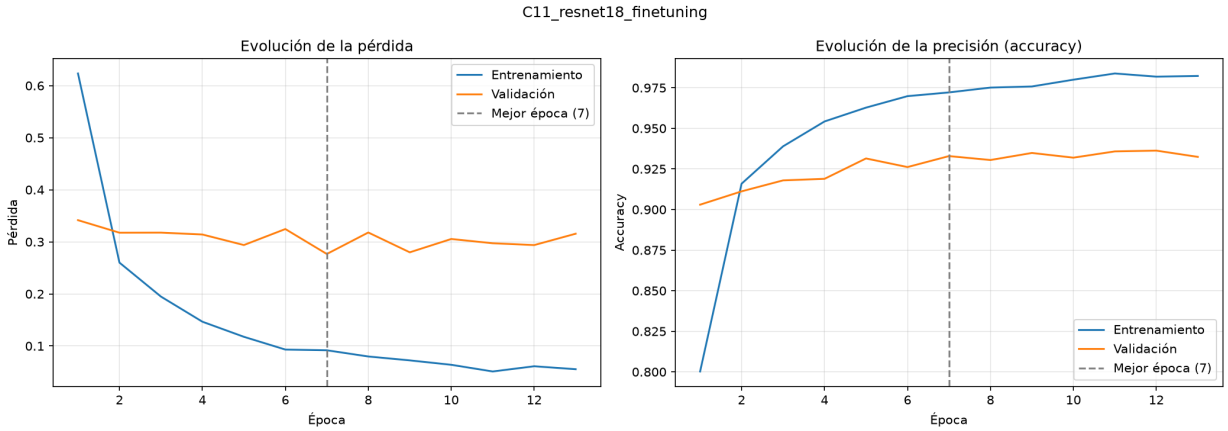


Figura 8. Curvas de C11 (ResNet18 con fine-tuning de layer4).

Criterio	C10 (CNN propia)	C11 (ResNet18)
Accuracy de validación	95,4 %	93,3 %
Pérdida de validación	0,221	0,277
Parámetros (entrenables)	4,59 M (todos)	11,18 M (8,40 M)
Épocas ejecutadas (mejor)	31 (23)	13 (7)
Tiempo total de entrenamiento	9,8 min	3,6 min
Tiempo medio por época	18,9 s	16,7 s

Tabla 6. Comparación entre la mejor red propia y la ResNet18. C10 es mejor y tiene menos de la mitad de parámetros; C11 llega antes a su mejor época.

7. Comparativa global y selección del modelo

Conf.	Estrategia	Parám.	Mejor ép.	Train Acc	Val Loss	Val Acc	Tiempo
C1	Manual	2,16 M	5/10	86,1 %	0,638	81,7 %	0,6 min
C2	Manual	2,16 M	6/11	84,0 %	0,567	82,9 %	0,6 min
C3	Manual	8,48 M	6/11	90,0 %	0,547	84,9 %	1,1 min
C4	Manual	4,59 M	8/13	93,3 %	0,405	89,7 %	1,2 min
C5	Manual	4,59 M	8/13	90,1 %	0,470	86,6 %	1,3 min
C6	Manual	4,59 M	8/13	96,1 %	0,405	88,8 %	1,3 min
C7	Manual	8,84 M	11/16	92,6 %	0,384	89,3 %	1,7 min
C8	Manual + aumento	4,59 M	14/22	95,0 %	0,258	93,3 %	6,8 min
C9	Manual + aumento	8,84 M	19/27	94,0 %	0,229	93,7 %	8,6 min
C10	Optuna + aumento	4,59 M	23/31	97,4 %	0,221	95,4 %	9,8 min
C11	ResNet18 + aumento	11,18 M	7/13	97,2 %	0,277	93,3 %	3,6 min

Tabla 7. Resumen de los 11 modelos entrenados (tiempos en una GPU NVIDIA con CUDA). En verde, el modelo seleccionado.

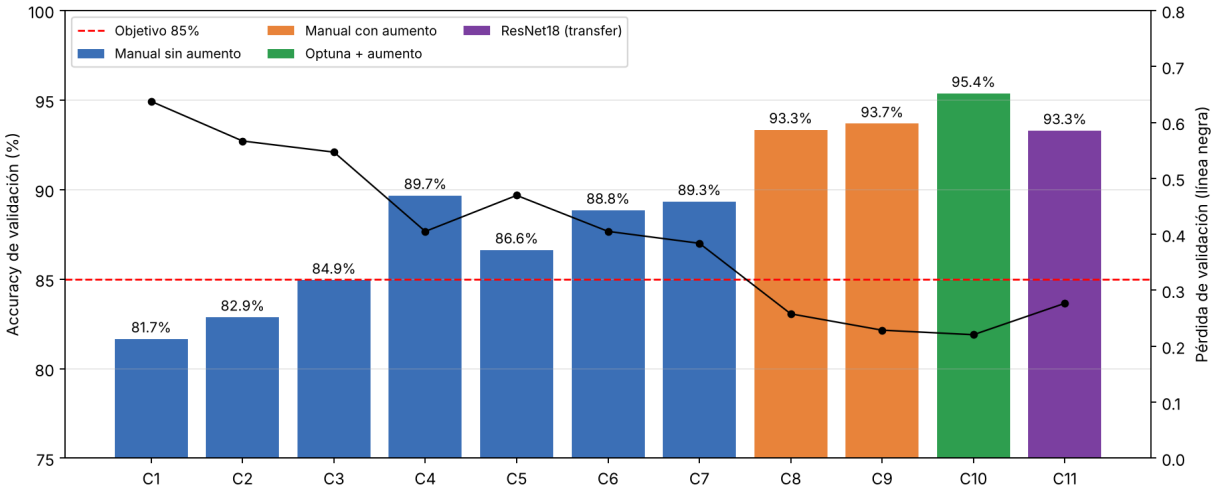


Figura 9. Accuracy (barras) y pérdida (línea negra) de validación de todas las configuraciones.

El modelo final se elige de forma automática como el de **menor pérdida de validación**, el mismo criterio que usa el early stopping. La pérdida es una medida más fina que la accuracy: dos modelos con la misma accuracy pueden tener pérdidas muy distintas si uno acierta con mucha más seguridad. En nuestro caso **C10 gana por los dos criterios**. Con 4,6 M de parámetros es además menos de la mitad de grande que C7, C9 o la ResNet18.

8. Evaluación final en el conjunto de test

El conjunto de test (496 imágenes) no se ha usado en ningún momento para elegir hiperparámetros ni modelos: se evalúa **una sola vez**, al final, con el modelo seleccionado (C10).

Personaje	Precision	Recall	F1	Imágenes
Homer Simpson	1,000	1,000	1,000	50
Ned Flanders	1,000	1,000	1,000	49
Moe Szyslak	1,000	1,000	1,000	50
Lisa Simpson	1,000	1,000	1,000	50
Bart Simpson	1,000	1,000	1,000	50
Marge Simpson	1,000	1,000	1,000	50
Krusty The Clown	1,000	1,000	1,000	50
Principal Skinner	1,000	1,000	1,000	50
Charles Montgomery Burns	1,000	1,000	1,000	48
Milhouse Van Houten	1,000	1,000	1,000	49
Accuracy global			**1,000**	**496**

*Tabla 8. Informe de clasificación de C10 en test: **accuracy 100 %**, pérdida 0,0117.*

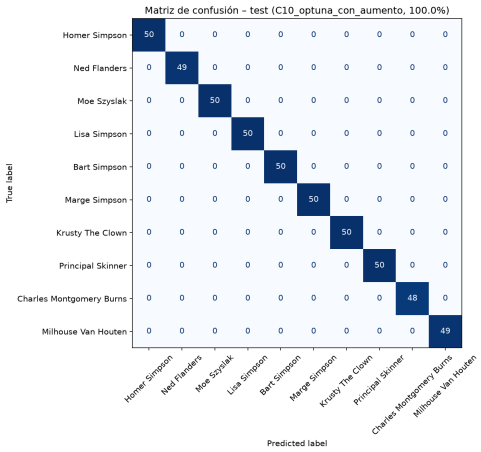


Figura 10. Matriz de confusión de C10 en el conjunto de test: las 496 imágenes se clasifican correctamente.

Por qué el test da un 100 % y la validación un 95,4 %

Un resultado perfecto en test, varios puntos por encima de validación, es sospechoso, así que lo investigamos. Comparamos cada imagen de test con las 11.739 de entrenamiento: reducimos todas a 16×16 en escala de grises y calculamos la correlación con la imagen de entrenamiento más parecida. Después comprobamos las parejas a resolución completa:

- **253 de las 496 imágenes de test (51 %)** tienen en entrenamiento una imagen prácticamente idéntica: la diferencia media es inferior a 3/255 por píxel, es decir, es el mismo fotograma con otra compresión JPEG. En todos los casos la pareja pertenece al mismo personaje.
- En total, alrededor del **84 %** del test tiene un «gemelo» casi exacto en entrenamiento (correlación > 0,99). En nuestro conjunto de validación, separado al azar del de entrenamiento, esta cifra es solo del 12 %.

Es decir, el test oficial de Kaggle se construyó con fotogramas que también están en el conjunto de entrenamiento publicado, y por eso es mucho más fácil que nuestra validación. No hay fuga de datos en nuestro código: el test se carga desde una carpeta distinta y solo se usa en la última celda. Pero la cifra del 100 % **sobreestima** la capacidad real del modelo para reconocer imágenes nuevas. La estimación honesta es la de validación, **95,4 %**, que aun así supera con claridad el objetivo del 85 %.

9. Conclusiones

- Se ha construido un clasificador de 10 personajes de Los Simpson con redes convolucionales, pooling, dropout y early stopping que cumple el objetivo de la práctica: **95,4 % en validación y 100 % en test**, frente al 85 % pedido.
- La búsqueda manual (7 configuraciones) mostró que la profundidad convolucional y un dropout alto son los factores clave, y pasó del 81,7 % al 89,7 %.
- El **aumentado de datos** fue la mejora más efectiva frente al sobreajuste: +4–5 puntos.
- La **búsqueda automática con Optuna** confirmó la arquitectura elegida a mano, encontró un dropout y una LR mejores y, combinada con aumento, dio el mejor modelo (C10).
- La **transferencia de conocimiento** con ResNet18 converge mucho más rápido, pero no supera a nuestra red: el dominio de dibujos animados está lejos de las fotografías de ImageNet.
- El test oficial comparte gran parte de sus fotogramas con el entrenamiento, por lo que tomamos la validación como medida fiable. Como mejoras futuras: un test con episodios distintos a los de entrenamiento, imágenes de mayor resolución para la ResNet o descongelar más capas con una LR menor.